

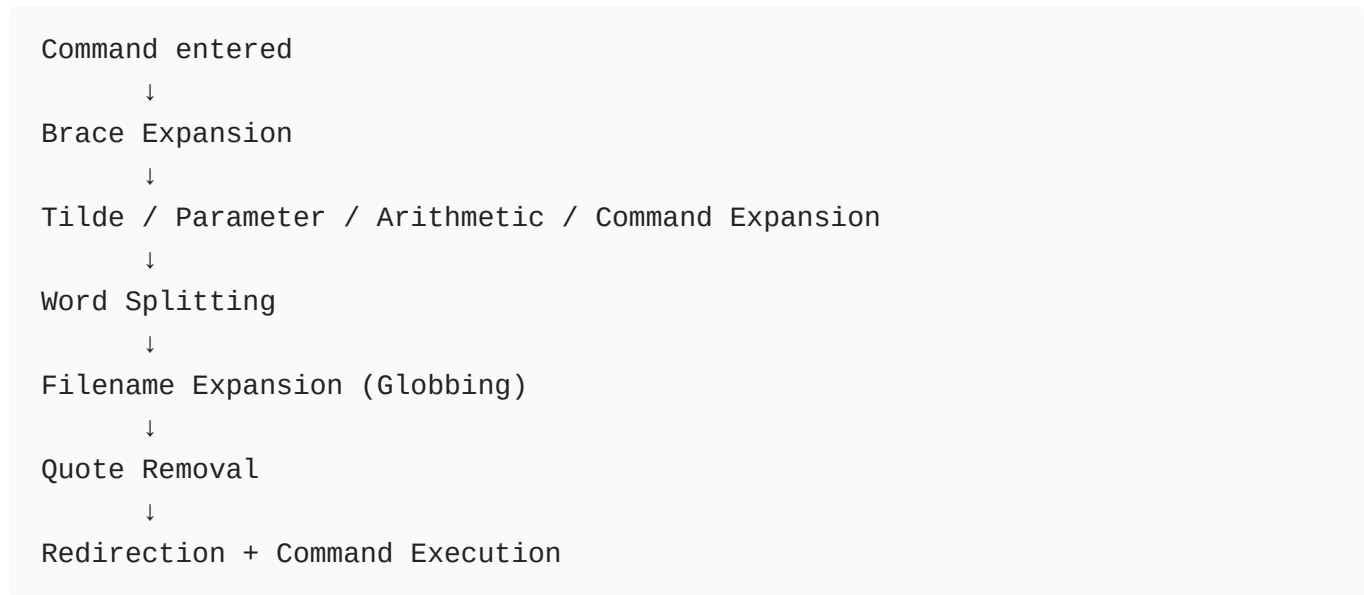
DAY 10

Bash Shell Expansions & Command Processing – Globbing, Quote Removal & Redirection

1. Where These Topics Fit

When Bash processes a command, it performs several transformations before actually executing it.

A simplified mental model is:



The important idea is:

Bash doesn't simply execute the text exactly as you typed it. It first interprets and transforms it.

2. Globbing – Filename Expansion

What is Globbing?

Globbing, also called **filename expansion** or **pathname expansion**, is the process where Bash uses special pattern characters to match filenames and directory names.

The most important globbing characters are:

Symbol	Meaning
*	Matches zero or more characters
?	Matches exactly one character
[]	Matches one character from a specified set/range

Example directory

Suppose we have:

```
notes.txt
notes.pdf
notes.md
report.txt
report.pdf
image.png
```

Then:

```
ls *.txt
```

can match:

```
notes.txt
report.txt
```

The shell expands:

```
*.txt
```

into the matching filenames **before** `ls` receives the arguments.

So conceptually:

```
ls *.txt
  ↓
ls notes.txt report.txt
```

3. The `*` Wildcard

Meaning

`*` matches **zero or more characters**.

For example:

```
ls *.txt
```

means:

Find filenames that end with `.txt`.

It can match:

```
a.txt
notes.txt
report.txt
my-long-report.txt
.txt
```

Notice that `*` can represent **nothing**, so `.txt` can technically match.

More examples

```
ls file*
```

Matches:

```
file
file.txt
file1
filename
file_backup
```

Because everything starts with:

```
file
```

```
ls *report*
```

Can match:

```
report.txt
security_report.pdf
myreport.md
report_backup
```

Because `report` appears somewhere in the filename.

```
ls report*.txt
```

Matches filenames that:

1. start with `report`
2. end with `.txt`

For example:

```
report.txt
report1.txt
report_final.txt
report_backup.txt
```

4. The `?` Wildcard

Meaning

`?` matches **exactly one character**.

For example:

```
ls file?.txt
```

could match:

```
file1.txt  
file2.txt  
fileA.txt
```

But not:

```
file10.txt
```

because `10` contains **two characters**, while `?` represents exactly one.

Compare

```
file?.txt
```

matches:

```
file1.txt  
fileA.txt  
file9.txt
```

while:

```
file*.txt
```

could match:

```
file.txt  
file1.txt  
file10.txt  
file123.txt
```

Easy memory trick

```
* → any number of characters  
? → exactly one character
```

5. Bracket Expansion / Character Classes `[]`

Square brackets allow you to specify a **set or range of characters**.

For example:

```
ls file[123].txt
```

can match:

```
file1.txt  
file2.txt  
file3.txt
```

But not:

```
file4.txt
```

Character ranges

You can specify ranges using `-`.

`[0-9]`

```
ls file[0-9].txt
```

Means:

| Match one character between `0` and `9`.

Possible matches:

```
file0.txt  
file1.txt  
file2.txt  
...  
file9.txt
```

[a-z]

```
ls [a-z].txt
```

Matches filenames whose relevant character is a lowercase letter from `a` through `z`.

[A-Z]

Matches uppercase letters.

Multiple characters

```
ls file[abc].txt
```

means:

| Match one character that is either `a`, `b`, or `c`.

So:

```
filea.txt  
fileb.txt  
filec.txt
```

can match.

6. Negating a Character Class

You can also specify characters that **should not** match.

For example:

```
[!0-9]
```

means:

| One character that is not a digit.

In Bash, `^` can also be used for negation in many glob patterns:

```
[^0-9]
```

But `[!0-9]` is the more conventional Bash form.

7. Globbing vs Regular Expressions

This is **very important**.

Globbing is **not the same thing** as regular expressions.

For example:

Globbing:

- `*` → any number of characters
- `?` → one character

In regular expressions:

Regex:

- `.*` → any number of characters
- `.` → one character

So don't automatically apply regex rules to Bash globbing.

8. Globbing Only Matches Existing Files?

A very important distinction:

Brace expansion

```
echo file{1,2,3}.txt
```

produces:

```
file1.txt file2.txt file3.txt
```

even if those files don't exist.

Globbing

```
echo file*.txt
```

attempts to match **actual directory entries** matching the pattern.

Therefore:

Brace expansion generates names. Globbing matches filesystem names.

This distinction is frequently useful in Bash scripting.

9. Practical Globbing Examples

Find all `.log` files

```
ls *.log
```

Find files beginning with `access`

```
ls access*
```

Find files beginning with `auth` and ending in `.log`

```
ls auth*.log
```

Find one-digit numbered files

```
ls file[0-9].txt
```

Find files with one arbitrary character

```
ls file?.txt
```

10. Security/SOC Relevance of Globbing

Globbing is extremely useful in security automation.

For example:

```
ls /var/log/*.log
```

could help identify log files.

Or:

```
grep "Failed password" /var/log/auth*.log
```

could search authentication-related logs.

You can also use patterns when processing:

- log files
- configuration files
- malware samples in a lab
- reports
- backup files
- temporary files
- security scan results

Important security consideration

Always understand whether a variable is being expanded safely.

For example, an unquoted expansion may undergo word splitting and pathname expansion.

So this:

```
rm $file
```

can behave differently from:

```
rm "$file"
```

if `$file` contains spaces or glob characters.

Quoting is therefore an important Bash security and reliability practice.

11. Quote Removal

After Bash performs the relevant expansions, it performs **quote removal**.

The purpose of quoting is to tell Bash:

"Treat this part of the command literally rather than interpreting it in the usual way."

Bash uses three important quoting mechanisms:

<code>'...'</code>	Single quotes
<code>"..."</code>	Double quotes
<code>\</code>	Backslash

12. Single Quotes `'...'`

Single quotes preserve the literal meaning of almost everything inside them.

Example:

```
echo '$HOME'
```

Output:

```
$HOME
```

Bash does **not** expand `$HOME` inside single quotes.

Compare:

```
echo "$HOME"
```

This expands the variable.

For example:

```
/home/sourya
```

So:

```
'$HOME' → literal text  
"$HOME" → variable expansion
```

13. Double Quotes "..."

Double quotes allow certain expansions while preventing ordinary word splitting and pathname expansion of the resulting expansion.

Example:

```
name="John Smith"  
echo "$name"
```

Output:

```
John Smith
```

The entire value is treated as one argument.

Without quotes:

```
echo $name
```

the expansion can undergo word splitting:

```
John  
Smith
```

as separate shell words/arguments.

14. Backslash \

A backslash can remove the special meaning of the next character.

For example:

```
echo \${HOME}
```

produces:

```
${HOME}
```

Instead of expanding `${HOME}`.

Another example:

```
echo "Hello \"World\""
```

produces:

```
Hello "World"
```

15. Why Quote Removal Exists

Suppose you write:

```
echo "Hello"
```

The quotes are needed by Bash to understand how the text should be treated.

But `echo` doesn't normally receive the quotes themselves as part of the argument.

Conceptually:

```
"Hello"
```

↓

```
Hello
```

The quotes served as instructions to the shell and are then removed.

This is the basic idea behind **quote removal**.

16. Redirection

Redirection controls **where a command gets its input from or where it sends its output**.

Normally:

```
Keyboard → STDIN → Program → STDOUT → Terminal
```

↓

```
STDERR
```

↓

```
Terminal
```

Redirection allows us to change these destinations/sources.

For example:

```
command > output.txt
```

Instead of displaying output on the terminal:

```
command
```

↓

```
output.txt
```

17. The Three Standard Streams

Every normal Unix/Linux process has three standard streams:

Stream	Number	Purpose	Default
STDIN	0	Input	Keyboard
STDOUT	1	Normal output	Terminal
STDERR	2	Error output	Terminal

Remember :

```
0 → input
1 → output
2 → error
```

This is extremely important for Linux administration and SOC automation.

18. STDIN – Standard Input

STDIN provides input to a program.

File descriptor:

```
0
```

Normally, it comes from the keyboard.

For example:

```
cat
```

`cat` waits for input from STDIN.

You can type:

```
Hello
```

and then provide EOF with:

```
Ctrl+D
```

19. STDOUT – Standard Output

STDOUT is normal program output.

File descriptor:

```
1
```

For example:

```
ls
```

normally sends its output to the terminal.

You can redirect it to a file:

```
ls > files.txt
```

Now:

```
ls
↓
STDOUT
↓
files.txt
```

20. > – Redirect STDOUT

Syntax:

```
command > file
```

Example:

```
ls > files.txt
```

The normal output goes into `files.txt`.

Important:

> **overwrites** the file if it already exists.

For example:

```
echo "Hello" > test.txt
```

Then:

```
echo "World" > test.txt
```

The file becomes:

```
World
```

The previous content was replaced.

21. >> – Append STDOUT

If you don't want to overwrite the file:

```
command >> file
```

Example:

```
echo "First line" > log.txt  
echo "Second line" >> log.txt
```

Result:

```
First line  
Second line
```

Memory trick

```
>   → replace  
>> → add to the end
```

22. STDERR – Standard Error

Errors are sent through:

```
STDERR = file descriptor 2
```

For example:

```
ls /does-not-exist
```

may produce an error such as:

```
ls: cannot access '/does-not-exist': No such file or directory
```

That message goes to **STDERR**, not **STDOUT**.

This distinction becomes very important when scripting.

23. Redirecting STDERR

Use:

```
2>
```

Example:

```
ls /does-not-exist 2> errors.txt
```

Now the error goes into:

```
errors.txt
```

instead of the terminal.

24. Redirect STDOUT and STDERR Separately

You can do:

```
command > output.txt 2> errors.txt
```

Now:

```
STDOUT → output.txt  
STDERR → errors.txt
```

This is extremely useful in automation.

For example:

```
./security_scan.sh > scan_output.txt 2> scan_errors.txt
```

You get:

```
Normal results → scan_output.txt  
Errors          → scan_errors.txt
```

25. Redirect Both STDOUT and STDERR

A common Bash syntax is:

```
command > output.txt 2>&1
```

Meaning:

```
STDOUT → output.txt  
STDERR → wherever STDOUT is currently going
```

So both go into:

```
output.txt
```

A shorter Bash-specific form is:

```
command &> output.txt
```

26. Order Matters in Redirection

This is a very important Bash concept.

Consider:

```
command > file 2>&1
```

This means:

```
STDOUT → file  
STDERR → same destination as STDOUT
```

So both go to `file`.

But:

```
command 2>&1 > file
```

is different because redirections are processed **from left to right**.

This is an important concept to understand as you become more advanced with Bash scripting.

27. Redirecting STDIN

You can also make a file provide input to a command.

Syntax:

```
command < file
```

Example:

```
cat < input.txt
```

Conceptually:

```
input.txt  
  ↓  
STDIN  
  ↓  
cat
```

Instead of reading from the keyboard, `cat` reads from the file.
